第一次汇报PPT

初版故事线

1. 从场景出发：为啥需要溯源 （为啥效果差，证据多、记忆转化、长输出）

1. 方法不适配
2. 分析根因（证据变化、RAG上下文证据少、输出也少，OOD）
3. 面向 harness，后处理，解耦

2. 具体的研究title 延伸

1. 需要 细粒度信息流 来解决问题（建模对象怎么来的）
2. 追踪agent记忆变化关系
3. 三个场景（DCC、溯源、归因）：展示推导链路，怎么来的；各自举一个case
    a. DCC展示逻辑：复杂规则任务中的最终判断，往往经过多次提取、解释、压缩和传递。我们需要知道每条判断依据哪些具体条款与事实，以及必要条件在传递过程中是否被保留下来

3. 挑战

1. 证据多、长输出，做好后处理（长）
2. 记忆转化压缩、子agent 调用如何跟踪证据链路（复杂hareness）

4. 方法创新点

1. 多证据长上下文 packing：放到一起、mask、一次传播（要画图）
2. 记忆转化，怎么flow起来：SP-hareness 轨迹森林 TITO 单挑到多条

5. 目前的效果

1. 多条genmem-bench效果
2. Flow： case study DCC

PPT批注（新版故事线）

第一二页【从场景出发：为啥需要溯源 （为啥效果差，证据多、记忆转化、长输出）】

方法不适配

分析根因（证据变化、RAG上下文证据少、输出也少，OOD）

面向 harness，后处理，解耦【解耦的问题】

场景需求：DCC 判断需要完整的依据链

北京大学 PEKING UNIVERSITY

任务要求：

读取文档与规则
核查相关事实和条件
给出引用、理由或判断

已有样例：CUAD 优先权类条款定位

任务：找出与 ROFR / ROFO / ROFN 类别相关的合同条款（转述）

实际返回的合同原句

2.1 Party A shall have a preemptive right to carry out mobile game services with Party B on a nationwide basis.

审阅者需要继续核对

依据来自**哪段材料**?
解释在**哪个中间环节**产生?

原文写了“优先权”，但类别关系需要进一步解释

对于复杂的DCC任务，只看系统输出不够

研究动机：判断依据需要可复核

北京大学 PEKING UNIVERSITY

DCC：Agent 阅读文档与规则，审阅者需要核对判断依据

合同核查问题
有优先权类条款吗?

→

实际输出节选
“Party A shall have a preemptive right … ”

原始依据是什么？**中间怎样解释**？**最终结果**保留了哪些信息？

一次可信的审阅，需要能沿着具体内容继续查下去

已有 CUAD 合同样例，问题为类别转述；此处展示来源核查需求，不判定条款的法律类别。

感觉好奇怪啊，感觉参考这个吧

- 为啥要做MemoryFlow因为组里的Agent需要可信
  - DCC需要可信的 证据证明
  - 报告生成需要 可信的引用
- 之前工作的问题
  - 艺峰的工作很NB，但是存在问题【写个引子就行，但是要给个非常直观做不了我们这活的原因】
  - 本质上再Agent场景下溯源这事儿有了更多的变体。更难解决了【表述自己优化一下】
    - 分析根因（证据变化、RAG上下文证据少、输出也少，OOD）
    - 面向 harness，后处理，解耦【解耦的问题】
  - 所以科学问题是如何在 Agent 系统下构建一个 通用的 可信溯源方法。【引出Flow】



第三四页【科学问题：Agent信息Flow】

- 那怎么构建这种可信溯源机制呢
  - 这个地方放case和 流的分析，看Agent的pipeline。引出来我们发现这个信息其实在Agent当中是一个流
  - 分析Case最好用DCC，如果没有用这个也行。

场景需求：DCC 判断需要完整的依据链

北京大学 PEKING UNIVERSITY

合同原文 · §2.1

"Party A shall have a **preemptive right** to carry out mobile game services with Party B on a nationwide basis."

约定"优先权"

研究摘要 · 记录节选

"**functionally analogous** to a right of first offer/negotiation"

"**No explicit** 'right of first refusal', 'right of first offer', or 'right of first negotiation' terminology appears …"

"… should be **reviewed by a lawyer**."

最终报告 · 原句摘录

"2.1 Party A shall have a **preemptive right** to carry out mobile game services with Party B on a nationwide basis."

可以看到，这里研究摘要增加了解释与保留意见，最终报告回填原句

因此，可审阅的对象不仅要包括原始条款，也应当包含处理过程中新增的解释

研究背景

北京大学 PEKING UNIVERSITY

用户需求
↓
需求理解 / 大纲规划
↓
检索文档 / evidence
↓
summary / observation
↓
风格化改写
↓
最终报告

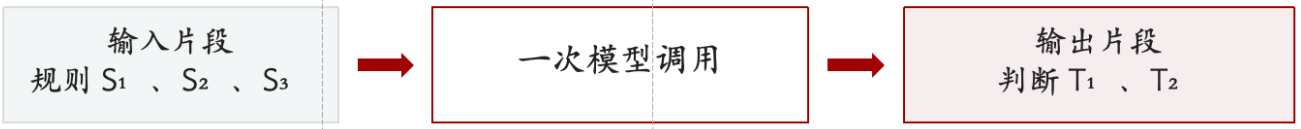
- 传统 RAG 通常将外部证据被直接用于生成回答，因此引用关系可以在"证据-输出"之间进行校验。
- 但在多智能体系统中，**证据信息会经过多轮中间状态传播**，报告生成为例：
  - 文档片段：原始证据形态
    - 【文档15】我们家的小康账本.docx：我们家的小康账本 核心阅读 记录柴米油盐…,现行标准下9899万农村贫困人口全部脱贫,832个贫困县全部摘帽,12.8万个贫困村全部出列…
  - Summary：压缩后的证据形态
    - …中国历史性解决绝对贫困问题，9899万农村贫困人口全部脱贫，832个贫困县摘帽，12.8万个贫困村出列。脱贫人口人均可支配收入从2021年的14051元增至2023年的16396元…
  - 报告：最终表达形态
    - 过去十年，咱们国家创造了人类减贫史上的奇迹。9899万农村贫困人口全部脱贫[1]，832个贫困县摘帽，12.8万个贫困村出列。脱贫人口人均可支配收入从2021年的14051元增长到2023年的16396元[2]…
  - 后续报告：风格变化，多轮对话
    - 诸君可知，这十年间，九千八百九十九万农村贫困人口，从"吃不饱穿不暖"到"衣食住行皆有保障"[1]。八百三十二个贫困县摘帽，十二点八万个贫困村出列…
- 随着信息载体和表达形态的不断变化，外部证据、中间记忆与最终输出之间的对应关系会逐渐变得隐式且难以追踪。
- 科学问题：如何在信息形态不断演化的多智能体系统中，维护外部证据、中间记忆与最终输出之间的可信归因关系？

思想自由 兼容并包

科学问题：对通用 Harness 信息流做建模

北京大学 PEKING UNIVERSITY

建模单位：TITO，也即一次模型调用的输入和输出



输出 × 输入	规则 S_1	规则 S_2	规则 S_3
判断 T_1	0.4317	0.0783	0.0147
判断 T_2	0.0647	0.3289	0.0693

行表示**输出单元**，列表示**可见来源**；每格记录一个回放来源分数

一次调用对应一张局部矩阵，一张调用矩阵可由多次前向得到，矩阵数量不等于前向次数

- 然后就引出来流了，那这玩意在 Agent领域场景是否是通用的
- 三个场景（DCC、溯源、归因）：展示推导链路，怎么来的；各自举一个case

DCC展示逻辑：复杂规则任务中的最终判断，往往经过多次提取、解释、压缩和传递。我们需要知道每条判断依据哪些具体条款与事实，以及必要条件在传递过程中是否被保留下来【感觉可以直接用下面的】

统一研究问题：细粒度信息怎样一步步传到结果

北京大学 PEKING UNIVERSITY

从DCC出发，但不止于DCC

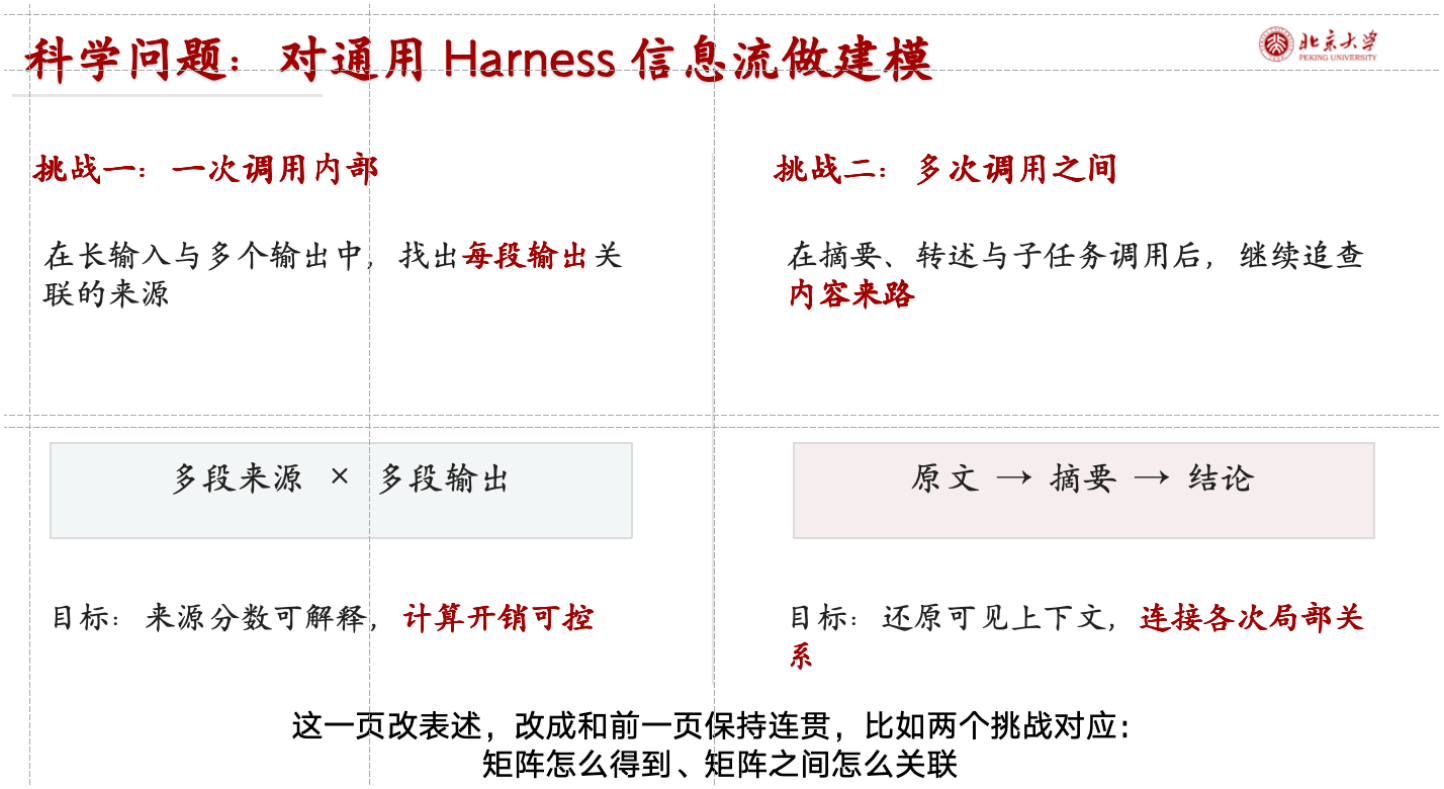
实际场景	沿链路追查什么	对应的细粒度内容
DCC 核查	判断依据与核查过程是否可查	条款条件、适用事实、逐项判断
引用溯源	输出经过哪些处理，来自哪些材料	原文片段、摘要子句、输出陈述
错误分析	可疑说法的出现和沿用位置	出错的说法、相关步骤与 Agent

维护从证据到判断的信息链路，才能逐项检查条件与内容变化

第五六页【引出挑战】

证据多、长输出，做好后处理（长），高效的解决引用问题

复杂Harness如何通用都可以用这套机制



这一页改表述，改成和前一页保持连贯，比如两个挑战对应：
矩阵怎么得到、矩阵之间怎么关联

第七页【引出方法】【写成方法总览】

针对挑战一。。基于艺峰，后处理，做成高效的prefill

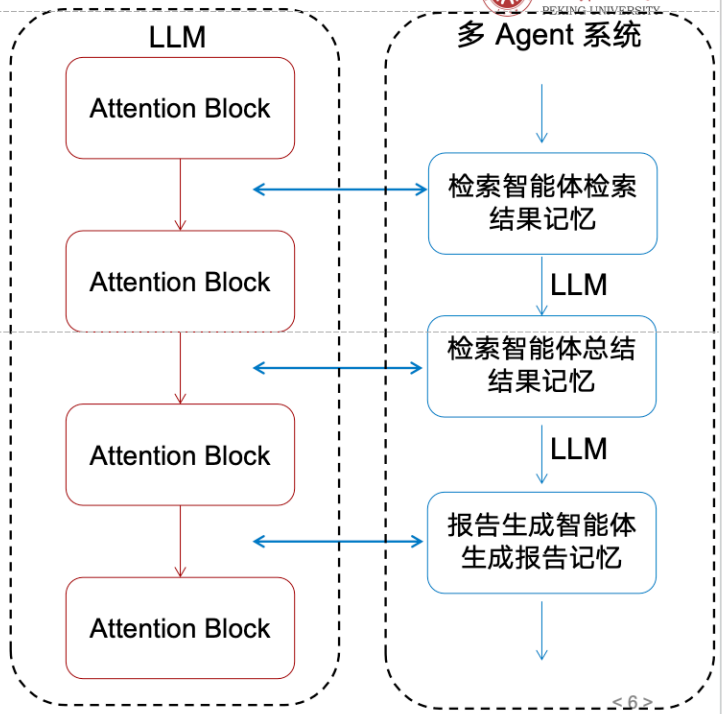
针对挑战二。。基于志心新科Harness RL

这个很烂，但是大体上是这样

直接说 基于艺峰和 志心新科的工作凝练出来的 可信机制

方法总览

- SP 视角上来看
  - 将整体的 Agent 系统变为一个 LLM 做类比
  - 中间的 Embedding 是对应记忆
  - Attention Block 对应 LLM API



思想自由 兼容并包

Scaffold Agent——方法总览

- **方法核心：**
  - 将报告生成视为一个不断演化的结构性框架，
- Pipeline
  - 中枢智能体决策根据注入 SOP 决策
    - 根据Utility分析报告瓶颈
    - 调用对应智能体Outline / Research / Reporter
  - 智能体执行反馈结果
  - 最终报告生成
- **智能体总览**
  - **中枢智能体**：全局决策与调度控制
  - **大纲智能体**：动态大纲维护
  - **检索智能体**：检索相关内容
  - **报告生成智能体**：报告生成



$$U_t = \lambda_1 U_{\text{ret}} + \lambda_2 U_{\text{str}} + \lambda_3 U_{\text{gen}},$$

思想自由 兼容并包

第八九页【具体方法for C1】

方法感觉表述有点抽象，但是大体内容没问题

方法一：用固定回放模型分析已有输出

保留注意力来源信号，将分析对象固定为已有文本

来源上下文 P
固定提示 + 候选材料

已有目标 T
本次需要解释的陈述

↓

冻结的 Qwen2.5-7B-Instruct

① 沿**预测目标词的位置**，读取固定头的注意力

② 对目标位置**取平均**，对候选来源区间**求和**，得到每个来源的分数

当前使用第 20 层第 16 个头，零基 (19,15)。固定模型与读出规则，不重新生成目标。

计算复用：相同长前缀只处理一次

逐目标重算

共同提示与材料 P → T1

共同提示与材料 P → T2

共同提示与材料 P → T3

每个目标都重读长材料

共享前缀后，分支依次处理

P 计算一次，保存结果

T1　T2　T3

公共计算复用，各目标保持独立

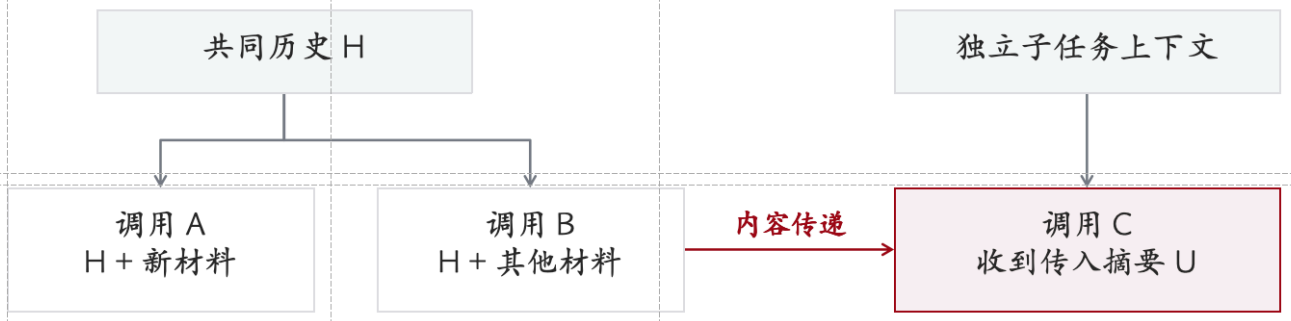
目标已知，可成块处理；完全相同的长前缀可以复用

第十十一页【具体方法for C2】

方法二：按实际上下文组织调用

共同历史形成分支，独立子任务形成新的上下文树

这一块讲的还是
不够清楚...

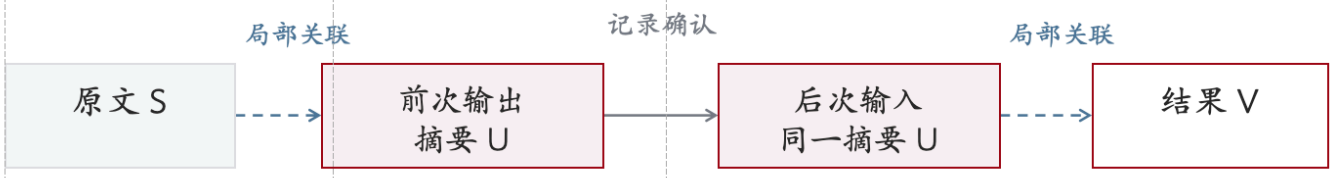


上下文结构限定可见范围，局部矩阵再描述具体来源关联

整体设计示意：固定案例已有请求与内容核对；通用会话森林、文件版本尚待完善。

多轮连接：同一内容接上前后矩阵

接点是实际传入的同一份内容



矩阵 1：摘要 U 对原文

矩阵 2：结果 V 对摘要 U 等输入

沿目标找到中间内容，再回查产生它的那次调用

遇到新的改写，就在产生改写的调用上建立新的局部关系

原文若也直接可见，保留直达路径。通用的路径聚合规则仍需验证。

感觉太抽象了

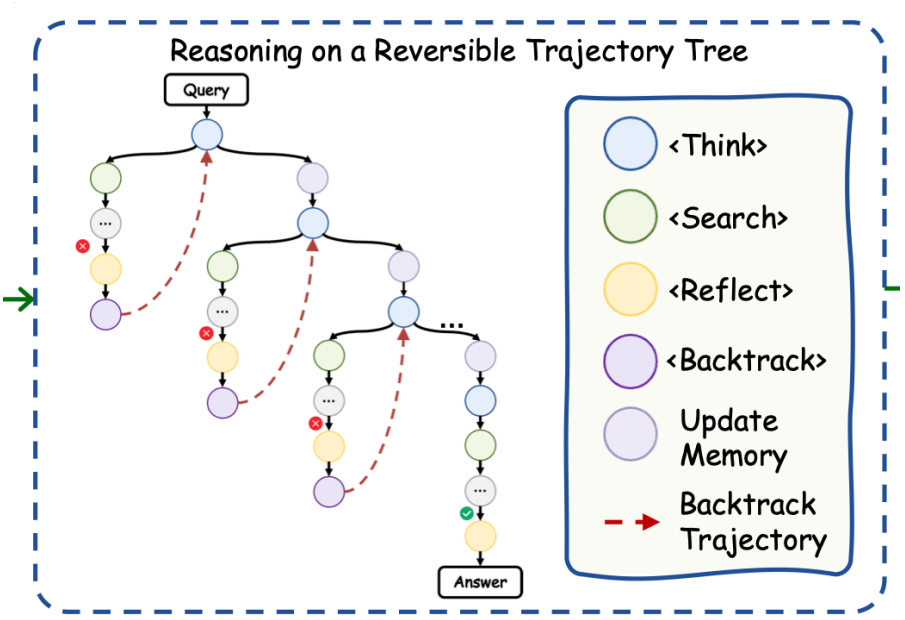
Agent轨迹是个森林【把子Agent 调用放进来，HarnessRL画进来】

边是 LLM调用

节点是Harness内部的状态

输出结果是最后一个节点

如果Harness是黑盒，Harness和LLM之间有交互，我们截取LLM和Harness交互，也就是调用LLM，是不是就知道了边，通过边去探测节点，探测 信息流。所以我们的边就是用我们上面的C1的矩阵来表示信息流。TITO来做好 这个流。



第十二页【实验】

A->B->C->D->E

等 A->B->C->D->E【唯一要多的时间，流水线。多10s时间】

单次的实验和SP的实验都要跑

准确率要拿下来，艺峰的准确率是下降的【跑GenMem】

艺峰是运行中生成，他是效率下降。我的方法在SP长轨迹上性能好。我们的方法可以跑更小的模型【试一下1.5b的模型】

统一候选与证据标准，主评测 N = 47

方法	完整证据@5	完整证据@10	首位片段命中
固定头回放（当前配置）	61.70%	70.21%	48.94%
DFAMS（迁移关系头）	53.19%	68.09%	34.04%
JEV	xx	xx	xx 加艺峰
BM25（词面检索）	55.32%	61.70%	57.45%
最后 8 层全头平均	57.45%	61.70%	34.04%

加速实验。。。。。

备选：生成时提取与独立回放的成本

单条短样例，同一生成服务；两条部署路线使用不同模型

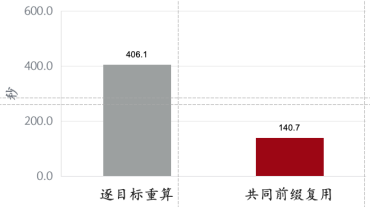
路线	稳态耗时	说明
只生成 S	3.687 s	正式 2 次，预热另列
生成 + 同模型来源提取	4.097 s	额外 0.410 s，正式 3 次
生成后 + 独立 7B 回放	4.701 s	回放 1.014 s，仅 1 次

此短条件下，生成时提取更快

输入 3,162 tokens，输出上限 128；生成 27B FP8，回放 7B。回放加载 9.179 s 另列；非同骨干算法比较。

计算效率：整批回放加速 2.89 倍

88 个目标 / 36 个前缀组，同模型、同输入、同评分



三遍耗时（秒）	重算	复用
1	536.87	140.72
2	405.40	140.94
3	406.09	140.61

中位加速 2.89×

逐候选分数完全一致
峰值显存均为 32.82 GiB

减少共同材料的重复计算，评分结果保持不变

正式三遍均保留。只计回放主体，加载、预处理与预热另列；共享 GPU，不等于整个 Agent 的加速。

DCC的case study

case study：DCC场景

展示单元：NBA RuleArena 中一个具体操作的合规判断

待补：完整 DCCv2 运行与真实内容

需要等梓涵师兄那边把DCCv2适配好

题面与规则	中间核查	最终判断
题面事实与假设原句 适用规则及条件原句 各自来源位置	Agent 如何解释规则 如何代入题面条件 相关请求与输出片段	实际判断及一句核心理由 采用的事实与规则 保留意见或冲突

同一目标的对照

只看最终引用：能核对哪些依据？
加入中间路径：能否定位解释、条件变化或冲突发生的位置？

G2 待补：完整运行记录、判断原句与来源材料。DCCv2 适配由协作方推进。